



OWL PRINT

CARDÁPIOS PERSONALIZADOS

Debug na Vercel + Variáveis de Ambiente

Guia detalhado de diagnóstico e configuração

Como ler erros, onde achar os logs e configurar cada variável · Junho/2026

PARTE A — Debug na Vercel (passo a passo)

"Debug" é o ato de **descobrir onde está o erro e por quê**. Na Vercel, quase tudo o que você precisa está nos **logs** (registros). Esta parte mostra onde encontrá-los e como entendê-los.

A.1 — Os status de um deploy (o que cada cor significa)

Toda vez que você envia código (push) ou clica em Redeploy, a Vercel cria um **deploy**. Ele passa por estados:

Status	Cor	O que significa	O que fazer
Queued	cinza	na fila, vai começar	esperar
Building	azul	construindo o site	esperar e acompanhar os logs
Ready	 verde	deu certo, está no ar	abrir o site
Error	 vermelho	falhou (erro de build)	abrir os Build Logs
Blocked	 vermelho	bloqueado antes de construir	ver A.4 ("Blocked")
Canceled	cinza	cancelado	refazer o deploy

A.2 — Onde ficam os logs (os dois tipos)

Existem **dois momentos** em que um erro acontece, e cada um tem seu log:

- 1) Erro de BUILD (construção) — antes do site existir** - É o erro mais comum no começo (ex.: erro de tipo do TypeScript). - **Onde ver:** menu **Deployments** → clique no deploy (linha) → você verá os **Build Logs** (o texto rolando que termina em "Ready" ou "Failed to compile").
- 2) Erro de RUNTIME (execução) — depois do site no ar** - Acontece quando alguém usa o site (ex.: o banco não conecta, uma página dá erro 500). - **Onde ver:** abra o seu projeto → aba **Logs** (em alguns layouts: **Observability** → **Logs** ou, dentro do deploy, **Runtime Logs / Functions**). Esses logs são em **tempo real** — deixe aberto e use o site para ver o erro aparecer.

Regra simples: **tela vermelha durante o deploy = Build Logs. Site no ar mas com erro = Runtime Logs.**

A.3 — Como LER um erro de build (anatomia)

Quando o build falha, a Vercel mostra algo assim:

```
Failed to compile.
./src/components/crm/ClientForm.tsx:39:47
Type error: 'res.data' is of type 'unknown'.

 37 |   const res = clientId ? await updateClient(...)
 38 |   if (!res.ok) return setServerError(res.error);
> 39 |   const id = clientId ?? ("data" in res ? res.data.id : "");
    |                                     ^
    |
```

Leia de cima para baixo: - **Failed to compile** → o build parou aqui. - **./src/components/crm/ClientForm.tsx:39:47** → o **arquivo** e a posição (**linha 39**, coluna 47). - **Type error: ...** → o **tipo** do problema (aqui, erro de tipagem do TypeScript). - **A seta > na linha 39** → a **linha exata** do erro. O  aponta o ponto.

🔗 Com isso você sabe **exatamente qual arquivo e linha** abrir. Se me mandar essa parte (print), eu corrijo o arquivo.

A.4 — Erros mais comuns e o que significam

Mensagem no log	O que é	Como resolver
<code>Failed to compile</code> + <code>Type error</code>	erro de tipagem (TypeScript)	corrigir o arquivo/linha indicado
<code>Module not found: Can't resolve 'X'</code>	import errado ou dependência faltando	conferir o caminho do import; rodar <code>npm install</code>
Deploy "Blocked"	o e-mail do commit não pertence à sua conta Vercel	<code>git config --global user.email "seu-email-da-vercel"</code> e novo commit
Build OK, site com 500 Internal Server Error	erro de runtime (banco, variável faltando)	ver Runtime Logs ; conferir variáveis
<code>MONGODB_URI is not defined</code> (ou similar)	variável de ambiente faltando na Vercel	adicionar a variável e Redeploy
<code>querySrv ECONNREFUSED ...mongodb.net</code>	DNS não resolve o banco	liberar IP no Atlas; (no PC) trocar DNS p/ 8.8.8.8

A.5 — A dica de ouro: teste o build no seu PC ANTES de subir

Esta única dica economiza **horas**. Antes de enviar para a Vercel, rode no terminal, na pasta do projeto:

```
npm run build
```

- Se aparecer "**Compiled successfully**" → o mesmo build vai passar na Vercel. Pode enviar.
- Se aparecer um **erro** → é **exatamente** o mesmo que apareceria na Vercel, só que em ~20 segundos e sem espera. Corrija, rode de novo, e só então faça o `git push`.

Em resumo: **nunca suba sem rodar** `npm run build` **localmente**. Você vê o erro antes da Vercel.

A.6 — Forçar um deploy novo / limpar cache

Quando você corrige algo e quer reconstruir: 1. **Deployments** → no deploy do topo, clique nos **três pontinhos** `...` → **Redeploy**. 2. Na janela, **desmarque** "Use existing Build Cache" para uma construção limpa. 3. Confirme em **Redeploy**.

A.7 — Conferir de qual commit é o deploy

Para ter certeza de que a Vercel está construindo o código **certo** (e não um antigo): - Abra o deploy → veja o **commit** (mensagem + código curto, ex.: `f976276`) e o horário. - Compare com o último commit no GitHub. Se forem diferentes, a Vercel não pegou seu envio (verifique se o `git push` foi para a branch **main**).

PARTE B — Variáveis de Ambiente (detalhado)

Variáveis de ambiente são as **configurações e senhas** do sistema (banco, login, imagens). Elas **não** ficam no código — ficam guardadas com segurança na Vercel.

B.1 — Onde adicionar na Vercel

- Abra seu **projeto** na Vercel → **Settings** → **Environment Variables**.
- Para cada variável: digite o **Name** (nome), o **Value** (valor), escolha os ambientes (marque os **três**: Production, Preview, Development) e clique em **Save**.
- Sempre que adicionar ou mudar uma variável, faça um Redeploy** (Parte A.6) — só assim ela passa a valer.

B.2 — Regras de ouro (evitam 90% dos erros)

- Sem aspas**. Cole o valor puro. Ex.: `MONGODB_URI` recebe `mongodb+srv://... , não "mongodb+srv://..."`.
- Sem espaços** no começo/fim do valor.
- `NEXT_PUBLIC_` **é PÚBLICO**. Tudo que começa com `NEXT_PUBLIC_` fica visível no navegador. **Nunca** coloque senha/segredo numa variável `NEXT_PUBLIC_`.
- Os segredos** (sem `NEXT_PUBLIC_`) ficam só no servidor — é onde vão as senhas.
- Mudou? Redeploy**. A Vercel não aplica variável nova sem reconstruir.

B.3 — As variáveis OBRIGATÓRIAS

Sem estas, o sistema não funciona:

Banco de dados

`MONGODB_URI` — endereço do banco no MongoDB Atlas. - Onde pegar: Atlas → cluster → **Connect** → **Drivers**. Troque `<password>` pela senha real e adicione `/owlprint` antes do `?`. - Exemplo: `mongodb+srv://renan:MinhaSenha123@cluster0.ab12c.mongodb.net/owlprint?retryWrites=true&w=majority` - Cuidado: senha com caractere especial (`@ # /`) quebra a string. Use senha só com letras e números.

Login do painel (Auth0) — segretos

`AUTH0_SECRET` — chave aleatória para criptografar a sessão. - Onde pegar: gere em **generate-secret.vercel.app/32** (ou `openssl rand -hex 32`). - Exemplo: `3f9c1a...` (32+ caracteres).

`AUTH0_BASE_URL` — o endereço do seu site. - No início (sem domínio): `https://owl-print.vercel.app` - Depois do domínio: `https://owlprintcardapios.com.br`

`AUTH0_ISSUER_BASE_URL` — o domínio do seu Auth0. - Onde pegar: Auth0 → sua aplicação → **Settings** → **Domain** (adicione `https://` na frente). - Exemplo: `https://dev-abc123.us.auth0.com`

`AUTH0_CLIENT_ID` — identificador da aplicação no Auth0 (em Settings).

`AUTH0_CLIENT_SECRET` — segredo da aplicação no Auth0 (em Settings).

Importante: no Auth0, em **Allowed Callback URLs**, coloque `SEU-SITE/api/auth/callback` e, em **Allowed Logout URLs**, coloque `SEU-SITE`. Troque para o domínio quando ele estiver pronto.

Imagens (Cloudinary)

`CLOUDINARY_CLOUD_NAME` — nome da sua "nuvem". (Dashboard do Cloudinary) `CLOUDINARY_API_KEY` — chave da API. (Dashboard) `CLOUDINARY_API_SECRET` — segredo da API. (Dashboard — secreto) `NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME` — o **mesmo** Cloud name acima (versão pública, usada no navegador). `NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET` — `owlprint_unsigned` (o preset **Unsigned** que você criou em Settings → Upload).

Site

`NEXT_PUBLIC_APP_URL` — endereço público do site. - Início: `https://owl-print.vercel.app` · Depois: `https://owlprintcardapios.com.br`

`NEXT_PUBLIC_WHATSAPP_NUMBER` — número do WhatsApp que recebe os contatos. - Formato: **só dígitos, com DDI 55**. Exemplo: `5511984721130` (sem `+`, sem espaços, sem traços).

B.4 — As variáveis OPCIONAIS (o site funciona sem elas)

Variável	Para quê	Exemplo
<code>NEXT_PUBLIC_GA_ID</code>	Google Analytics 4	<code>G-XXXXXXXXXX</code>
<code>NEXT_PUBLIC_GTM_ID</code>	Google Tag Manager	<code>GTM-XXXXXXX</code>
<code>NEXT_PUBLIC_META_PIXEL_ID</code>	Meta (Facebook) Pixel	<code>123456789012345</code>
<code>RESEND_API_KEY</code>	enviar orçamento por e-mail	<code>re_xxxxxxxx</code>
<code>EMAIL_FROM</code>	remetente do e-mail	<code>OWL PRINT <orcamentos@seudominio.com.br></code>
<code>AUTH0_AUDIENCE</code>	API do Auth0 (avanzado)	normalmente vazio

Pode deixar essas **em branco** ou nem criar. O sistema ignora com segurança.

B.5 — Tabela-resumo (copie na ordem)

Variável	Obrigatória?	Pública?
MONGODB_URI	✔ Sim	🔒 Secreta
AUTH0_SECRET	✔ Sim	🔒 Secreta
AUTH0_BASE_URL	✔ Sim	🔒 Secreta
AUTH0_ISSUER_BASE_URL	✔ Sim	🔒 Secreta
AUTH0_CLIENT_ID	✔ Sim	🔒 Secreta
AUTH0_CLIENT_SECRET	✔ Sim	🔒 Secreta
CLOUDINARY_CLOUD_NAME	✔ Sim	🔒 Secreta
CLOUDINARY_API_KEY	✔ Sim	🔒 Secreta
CLOUDINARY_API_SECRET	✔ Sim	🔒 Secreta
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME	✔ Sim	🌐 Pública
NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET	✔ Sim	🌐 Pública
NEXT_PUBLIC_APP_URL	✔ Sim	🌐 Pública
NEXT_PUBLIC_WHATSAPP_NUMBER	✔ Sim	🌐 Pública
NEXT_PUBLIC_GA_ID	☐ Opcional	🌐 Pública
NEXT_PUBLIC_GTM_ID	☐ Opcional	🌐 Pública
NEXT_PUBLIC_META_PIXEL_ID	☐ Opcional	🌐 Pública
RESEND_API_KEY	☐ Opcional	🔒 Secreta
EMAIL_FROM	☐ Opcional	🔒 Secreta

B.6 — Erros comuns ligados a variáveis

Sintoma	Causa	Solução
Login do admin não abre / erro de callback	AUTH0_BASE_URL ou callback URL erradas	igualar URL na Vercel e no Auth0; Redeploy
Site no ar mas dados não carregam	MONGODB_URI errada ou IP não liberado	conferir a string; liberar 0.0.0.0/0 no Atlas
Upload de imagem falha	preset não é Unsigned ou nome diferente	conferir NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET
Mudei a variável e não mudou nada	faltou reconstruir	Redeploy após salvar
Botão WhatsApp abre número errado	formato do número	usar só dígitos com DDI: 5511...